

```
!pip install datasets
!pip install tensorflow
```

```
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0,>=2023.1.0) (2025.3.0)
Requirement already satisfied: huggingface-hub<=0.24.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.36.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets) (25.0)
Requirement already satisfied: pyyaml<=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets) (6.0.3)
Requirement already satisfied: aiohttp!=4.0.0a0,!4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0,>=2023.1.0) (4.0.0a1)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<=0.24.0) (4.12.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<=0.24.0) (1.1.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (3.3.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (2.3.1)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (2025.11.11)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (2.5.0)
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (25.3.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (1.5.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (6.3.0)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (0.2.0)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (1.18.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->datasets) (1.17.0)
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.19.0)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.9.23)
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.0)
Requirement already satisfied: protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<6.0.0dev,>=3.20.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.21.5)
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.2.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15.0)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.1)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.76.0)
Requirement already satisfied: tensorboard~=2.19.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.19.0)
Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.10.0)
Requirement already satisfied: numpy<2.2.0,>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.15.1)
Requirement already satisfied: ml-dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.3)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensorflow) (0.45.1)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.17.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.3.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2.3.1)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2025.11.11)
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorflow) (3.7.0)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorflow) (0.7.0)
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorflow) (3.1.0)
Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from werkzeug>=1.0.1->tensorboard~=2.19.0->tensorflow) (3.0.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.5.0->tensorflow) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.5.0->tensorflow) (2.19.0)
```

```
import tensorflow as tf
from tensorflow.keras.applications import EfficientNetB3
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras import layers, models
from datasets import load_dataset
import numpy as np
import matplotlib.pyplot as plt
```

```
dataset = load_dataset("Soh1Health/enhanced-indian-food-classification")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret 'HF_TOKEN' does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
```

```
warnings.warn(
```

```
dataset
```

```
DatasetDict({
  train: Dataset({
    features: ['image', 'label'],
    num_rows: 13311
  })
  validation: Dataset({
    features: ['image', 'label'],
    num_rows: 1067
  })
  test: Dataset({
    features: ['image', 'label'],
    num_rows: 1026
  })
})
```

```
dataset = load_dataset("SohlHealth/enhanced-indian-food-classification")
```

```
# Target per split
TRAIN_SIZE = 6000
VAL_SIZE = 1000
TEST_SIZE = 1000

# Buat dataset yang diperkecil
small_train = dataset["train"].shuffle(seed=42).select(range(TRAIN_SIZE))
small_val = dataset["validation"].shuffle(seed=42).select(range(VAL_SIZE))
small_test = dataset["test"].shuffle(seed=42).select(range(TEST_SIZE))

print(len(small_train), len(small_val), len(small_test))
```

```
6000 1000 1000
```

```
dataset["train"] = small_train
dataset["validation"] = small_val
dataset["test"] = small_test
```

```
total = len(dataset["train"]) + len(dataset["validation"]) + len(dataset["test"])
total
```

```
8000
```

```
IMG_SIZE = 300

def process(batch):
    images = [img.convert("RGB").resize((IMG_SIZE, IMG_SIZE)) for img in batch["image"]]
    images = np.array([np.array(img)/255.0 for img in images])
    labels = np.array(batch["label"])
    return {"image": images, "label": labels}

train = dataset["train"].map(process, batched=True)
val = dataset["validation"].map(process, batched=True)
test = dataset["test"].map(process, batched=True)
```

```
-----
NameError                                Traceback (most recent call last)
/tmp/ipython-input-1997610245.py in <cell line: 0>()
      7     return {"image": images, "label": labels}
      8
----> 9 train = dataset["train"].map(process, batched=True)
     10 val = dataset["validation"].map(process, batched=True)
     11 test = dataset["test"].map(process, batched=True)

NameError: name 'dataset' is not defined
```

Next steps: [Explain error](#)

```
import tensorflow as tf
from tensorflow.keras.applications import EfficientNetB3
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras import layers, models
from datasets import load_dataset
import numpy as np
import matplotlib.pyplot as plt
```

```
dataset = load_dataset("SohlHealth/enhanced-indian-food-classification")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
```

```
TRAIN_SIZE = 6000
VAL_SIZE = 1000
TEST_SIZE = 1000

small_train = dataset["train"].shuffle(seed=42).select(range(TRAIN_SIZE))
small_val = dataset["validation"].shuffle(seed=42).select(range(VAL_SIZE))
small_test = dataset["test"].shuffle(seed=42).select(range(TEST_SIZE))

dataset["train"] = small_train
dataset["validation"] = small_val
dataset["test"] = small_test

print("Train :", len(dataset["train"]))
print("Val  :", len(dataset["validation"]))
print("Test  :", len(dataset["test"]))
```

```
Train : 6000
Val   : 1000
Test  : 1000
```

```
import tensorflow as tf
from tensorflow.keras.applications import EfficientNetB3
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras import layers, models
from datasets import load_dataset
import numpy as np
import matplotlib.pyplot as plt
```

```
dataset = load_dataset("SohlHealth/enhanced-indian-food-classification")
dataset
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
DatasetDict({
  train: Dataset({
    features: ['image', 'label'],
    num_rows: 13311
  })
  validation: Dataset({
    features: ['image', 'label'],
    num_rows: 1067
  })
  test: Dataset({
    features: ['image', 'label'],
    num_rows: 1026
  })
})
```

```

TRAIN_SIZE = 6000
VAL_SIZE = 1000
TEST_SIZE = 1000

dataset["train"] = dataset["train"].shuffle(seed=42).select(range(TRAIN_SIZE))
dataset["validation"] = dataset["validation"].shuffle(seed=42).select(range(VAL_SIZE))
dataset["test"] = dataset["test"].shuffle(seed=42).select(range(TEST_SIZE))

print(len(dataset["train"]), len(dataset["validation"]), len(dataset["test"]))

```

```
6000 1000 1000
```

```

IMG_SIZE = 300

def process(batch):
    images = [img.convert("RGB").resize((IMG_SIZE, IMG_SIZE)) for img in batch["image"]]
    images = np.array([np.array(img) / 255.0 for img in images])
    labels = np.array(batch["label"])
    return {"image": images, "label": labels}

train = dataset["train"].map(process, batched=True, batch_size=64)
val = dataset["validation"].map(process, batched=True, batch_size=64)
test = dataset["test"].map(process, batched=True, batch_size=64)

```

```

Map: 100%                               6000/6000 [04:10<00:00, 32.99 examples/s]
Map: 100%                               1000/1000 [01:19<00:00, 23.06 examples/s]
Map: 100%                               1000/1000 [01:28<00:00, 23.17 examples/s]

```

```

train_ds = tf.data.Dataset.from_tensor_slices((train["image"], train["label"])).shuffle(1000).batch(32)
val_ds = tf.data.Dataset.from_tensor_slices((val["image"], val["label"])).batch(32)
test_ds = tf.data.Dataset.from_tensor_slices((test["image"], test["label"])).batch(32)

```

```

a = []
while True:
    a.append('A' * 10_000_000)

```

```

!git lfs install
!git clone https://huggingface.co/datasets/SohlHealth/enhanced-indian-food-classification

```

```

Git LFS initialized.
Cloning into 'enhanced-indian-food-classification'...
remote: Enumerating objects: 29, done.
remote: Total 29 (delta 0), reused 0 (delta 0), pack-reused 29 (from 1)
Unpacking objects: 100% (29/29), 11.24 KiB | 1.87 MiB/s, done.
Filtering content: 100% (5/5), 1.95 GiB | 31.14 MiB/s, done.

```

```
!ls -R enhanced-indian-food-classification
```

```

enhanced-indian-food-classification:
data dataset_info.json README.md statistics

enhanced-indian-food-classification/data:
test-00000-of-00001.parquet  train-00002-of-00003.parquet
train-00000-of-00003.parquet  validation-00000-of-00001.parquet
train-00001-of-00003.parquet

enhanced-indian-food-classification/statistics:
dataset_stats.json

```

```
!pip install pyarrow pillow tqdm
```

```

Requirement already satisfied: pyarrow in /usr/local/lib/python3.12/dist-packages (18.1.0)
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (11.3.0)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (4.67.1)

```

```
import os

output_dir = "/content/IndianFoods"
os.makedirs(output_dir, exist_ok=True)
```

```
import pyarrow.parquet as pq
import os
from PIL import Image
import io
from tqdm import tqdm

input_dir = "/content/enhanced-indian-food-classification/data"
output_dir = "/content/IndianFoods"

os.makedirs(output_dir, exist_ok=True)

# Loop semua file parquet
for file in os.listdir(input_dir):
    if file.endswith(".parquet"):
        print("Processing:", file)
        table = pq.read_table(os.path.join(input_dir, file))
        df = table.to_pandas()

        for idx, row in tqdm(df.iterrows(), total=len(df)):
            img_bytes = row["image"]["bytes"]
            label = row["label"]

            # Nama kelas sebagai folder
            class_dir = os.path.join(output_dir, str(label))
            os.makedirs(class_dir, exist_ok=True)

            # Save image
            img = Image.open(io.BytesIO(img_bytes)).convert("RGB")
            img_path = os.path.join(class_dir, f"{file}-{idx}.jpg")
            img.save(img_path)
```

```
Processing: train-00000-of-00003.parquet
100%|██████████| 4437/4437 [00:37<00:00, 117.78it/s]
Processing: train-00002-of-00003.parquet
100%|██████████| 4437/4437 [00:49<00:00, 90.47it/s]
Processing: validation-00000-of-00001.parquet
100%|██████████| 1067/1067 [00:21<00:00, 50.57it/s]
Processing: test-00000-of-00001.parquet
100%|██████████| 1026/1026 [00:22<00:00, 45.72it/s]
Processing: train-00001-of-00003.parquet
100%|██████████| 4437/4437 [00:39<00:00, 112.10it/s]
```

```
!ls -R /content/IndianFoods
```

```

train-00001-of-00003.parquet-2529.jpg
train-00001-of-00003.parquet-2530.jpg
train-00001-of-00003.parquet-2531.jpg
train-00001-of-00003.parquet-2532.jpg
train-00001-of-00003.parquet-2533.jpg
train-00001-of-00003.parquet-2534.jpg
train-00001-of-00003.parquet-2535.jpg
train-00001-of-00003.parquet-2536.jpg
train-00001-of-00003.parquet-2537.jpg
train-00001-of-00003.parquet-2538.jpg
train-00001-of-00003.parquet-2539.jpg
train-00001-of-00003.parquet-2540.jpg
train-00001-of-00003.parquet-2541.jpg
train-00001-of-00003.parquet-2542.jpg
train-00001-of-00003.parquet-2543.jpg
train-00001-of-00003.parquet-2544.jpg
train-00001-of-00003.parquet-2545.jpg
train-00001-of-00003.parquet-2546.jpg
train-00001-of-00003.parquet-2547.jpg
train-00001-of-00003.parquet-2548.jpg
train-00001-of-00003.parquet-2549.jpg
train-00001-of-00003.parquet-2550.jpg
train-00001-of-00003.parquet-2551.jpg
train-00001-of-00003.parquet-2552.jpg
train-00001-of-00003.parquet-2553.jpg
train-00001-of-00003.parquet-2554.jpg
train-00001-of-00003.parquet-2555.jpg
train-00001-of-00003.parquet-2556.jpg
train-00001-of-00003.parquet-2557.jpg
train-00001-of-00003.parquet-2558.jpg
train-00001-of-00003.parquet-2559.jpg

```

```

import tensorflow as tf
from tensorflow.keras import layers

```

```
!ls /content/IndianFoods
```

aloo_gobi	chole	green_chutney	lassi	pizza
aloo_masala	chole_bhature	gulab_jamun	masala_dosa	poha
bhindi_masala	coconut_chutney	idli	modak	rajma_curry
biryani	dal_makhani	jalebi	momos	ras_malai
burger	dhokla	kaathi_rolls	mutton_curry	samosa
butter_naam	dum_aloo	kadai_paneer	paani_puri	shahi_paneer
chai	fish_curry	kebab	pakode	white_rice
chapati	fried_rice	kheer	palak_paneer	
chicken_curry	ghevar	kulfi	pav_bhaji	

```

import tensorflow as tf
from tensorflow.keras import layers

```

```

train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    "/content/IndianFoods",
    labels="inferred",
    label_mode="int",
    image_size=(300,300),
    validation_split=0.2,
    subset="training",
    seed=42,
    batch_size=32
)

val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    "/content/IndianFoods",
    labels="inferred",
    label_mode="int",
    image_size=(300,300),
    validation_split=0.2,
    subset="validation",
    seed=42,
    batch_size=32
)

```

```

Found 15404 files belonging to 43 classes.
Using 12324 files for training.
Found 15404 files belonging to 43 classes.

```

```
Using 3080 files for validation.
```

```
!ls /content/IndianFoods
```

```
aloo_gobi      chole      green_chutney  lassi      pizza
aloo_masala    chole_bhature  gulab_jamun   masala_dosa  poha
bhindi_masala  coconut_chutney  idli          modak        rajma_curry
biryani        dal_makhani    jalebi        momos        ras_malai
burger         dhokla        kaathi_rolls  mutton_curry  samosa
butter_naam    dum_aloo      kadai_paneer  paani_puri   shahi_paneer
chai           fish_curry     kebab         pakode       white_rice
chapati        fried_rice     kheer         palak_paneer
chicken_curry  ghevar        kulfi         pav_bhaji
```

```
allowed_classes = [
    "aloo_gobi", "aloo_masala", "bhindi_masala", "biryani", "burger", "butter_naam",
    "chai", "chapati", "chicken_curry", "chole_bhature", "dal_makhani",
    "dhokla", "fried_rice", "idli", "jalebi", "kaathi_rolls", "kadai_paneer", "kulfi",
    "masala_dosa", "momos", "paani_puri", "pakode", "pav_bhaji", "pizza", "samosa"
]
```

```
import os, shutil
```

```
source = "/content/IndianFoods"
destination = "/content/IndianFoods25"
os.makedirs(destination, exist_ok=True)
```

```
for cls in allowed_classes:
    src = os.path.join(source, cls)
    dst = os.path.join(destination, cls)
    if os.path.exists(src):
        shutil.copytree(src, dst)
```

```
!ls /content/IndianFoods
```

```
aloo_gobi      chole      green_chutney  lassi      pizza
aloo_masala    chole_bhature  gulab_jamun   masala_dosa  poha
bhindi_masala  coconut_chutney  idli          modak        rajma_curry
biryani        dal_makhani    jalebi        momos        ras_malai
burger         dhokla        kaathi_rolls  mutton_curry  samosa
butter_naam    dum_aloo      kadai_paneer  paani_puri   shahi_paneer
chai           fish_curry     kebab         pakode       white_rice
chapati        fried_rice     kheer         palak_paneer
chicken_curry  ghevar        kulfi         pav_bhaji
```

```
allowed_classes = [
    "aloo_gobi", "aloo_masala", "bhindi_masala", "biryani", "burger", "butter_naam",
    "chai", "chapati", "chicken_curry", "chole_bhature", "dal_makhani",
    "dhokla", "fried_rice", "idli", "jalebi", "kaathi_rolls", "kadai_paneer", "kulfi",
    "masala_dosa", "momos", "paani_puri", "pakode", "pav_bhaji", "pizza", "samosa"
]
```

```
!rm -rf /content/IndianFoods25
```

```
import os
import shutil
```

```
source = "/content/IndianFoods"
destination = "/content/IndianFoods25"
os.makedirs(destination, exist_ok=True)
```

```
for cls in allowed_classes:
    src = os.path.join(source, cls)
    dst = os.path.join(destination, cls)
    if os.path.exists(src):
        shutil.copytree(src, dst, dirs_exist_ok=True)
```

```
!ls /content/IndianFoods25
```

```
aloo_gobi      butter_naan    dal_makhani    kaathi_rolls   paani_puri
aloo_masala    chai           dhokla         kadai_paneer    pakode
bhindi_masala  chapati        fried_rice     kulfi           pav_bhaji
biryani         chicken_curry  idli           masala_dosa     pizza
burger          chole_bhature  jalebi         momos           samosa
```

```
import glob
import random
import shutil
import os

src = "/content/IndianFoods25"
dst = "/content/IndianFoods8000"
os.makedirs(dst, exist_ok=True)

max_images = 8000
all_images = []

# Ambil semua gambar dari 25 kelas
for cls in os.listdir(src):
    cls_folder = os.path.join(src, cls)
    for img in glob.glob(cls_folder + "/*.jpg"):
        all_images.append((img, cls))

# Acak semua gambar
random.shuffle(all_images)

# Pilih hanya 8000
selected = all_images[:max_images]

# Salin gambar ke folder baru
for img_path, cls in selected:
    out_folder = os.path.join(dst, cls)
    os.makedirs(out_folder, exist_ok=True)
    shutil.copy(img_path, out_folder)
```

```
!find /content/IndianFoods8000 -type f | wc -l
```

```
8000
```

```
!ls /content/IndianFoods8000
```

```
aloo_gobi      butter_naan    dal_makhani    kaathi_rolls   paani_puri
aloo_masala    chai           dhokla         kadai_paneer    pakode
bhindi_masala  chapati        fried_rice     kulfi           pav_bhaji
biryani         chicken_curry  idli           masala_dosa     pizza
burger          chole_bhature  jalebi         momos           samosa
```

```
import tensorflow as tf

train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    "/content/IndianFoods8000",
    validation_split=0.2,
    subset="training",
    seed=42,
    image_size=(300, 300),
    batch_size=32
)

val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    "/content/IndianFoods8000",
    validation_split=0.2,
    subset="validation",
    seed=42,
    image_size=(300, 300),
    batch_size=32
)
```

```
Found 8000 files belonging to 25 classes.
Using 6400 files for training.
Found 8000 files belonging to 25 classes.
Using 1600 files for validation.
```

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.applications import EfficientNetB3
```

```
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
])
```

```
base_model = EfficientNetB3(
    include_top=False,
    weights="imagenet",
    input_shape=(300,300,3)
)

base_model.trainable = False # freeze dulu
```

```
Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb3\_notop.h5
43941136/43941136 ————— 3s 0us/step
```

```
import os
import shutil
import random
from glob import glob

src = "/content/IndianFoods25"
dst = "/content/IndianFoodsBalanced"
target_count = 320

os.makedirs(dst, exist_ok=True)

for cls in sorted(os.listdir(src)):
    cls_src = os.path.join(src, cls)
    cls_dst = os.path.join(dst, cls)
    os.makedirs(cls_dst, exist_ok=True)

    # ambil semua gambar
    images = glob(cls_src + "/*.jpg")
    random.shuffle(images)

    if len(images) >= target_count:
        # pakai 320 yang pertama
        selected = images[:target_count]
    else:
        # jika kurang → duplikasi
        selected = images.copy()
        while len(selected) < target_count:
            selected.append(random.choice(images))

    # salin semua gambar ke folder balanced
    for i, img_path in enumerate(selected):
        filename = f"{cls}_{i}.jpg"
        shutil.copy(img_path, os.path.join(cls_dst, filename))

    print(f"{cls}: {len(selected)} gambar")
```

```
aloo_gobi: 320 gambar
aloo_masala: 320 gambar
bhindi_masala: 320 gambar
biryani: 320 gambar
burger: 320 gambar
butter_naam: 320 gambar
chai: 320 gambar
chapati: 320 gambar
chicken_curry: 320 gambar
chole_bhature: 320 gambar
```

```
dal_makhani: 320 gambar
dhokla: 320 gambar
fried_rice: 320 gambar
idli: 320 gambar
jalebi: 320 gambar
kaathi_rolls: 320 gambar
kadai_paneer: 320 gambar
kulfi: 320 gambar
masala_dosa: 320 gambar
momos: 320 gambar
paani_puri: 320 gambar
pakode: 320 gambar
pav_bhaji: 320 gambar
pizza: 320 gambar
samosa: 320 gambar
```

```
!find /content/IndianFoodsBalanced -type f | wc -l
```

```
8000
```

```
train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    "/content/IndianFoodsBalanced",
    validation_split=0.2,
    subset="training",
    seed=42,
    image_size=(300,300),
    batch_size=32
)

val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    "/content/IndianFoodsBalanced",
    validation_split=0.2,
    subset="validation",
    seed=42,
    image_size=(300,300),
    batch_size=32
)
```

```
Found 8000 files belonging to 25 classes.
Using 6400 files for training.
Found 8000 files belonging to 25 classes.
Using 1600 files for validation.
```

```
import tensorflow as tf
from tensorflow.keras import layers
from tensorflow.keras.applications import EfficientNetB3
```

```
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
])
```

```
base_model = EfficientNetB3(
    include_top=False,
    weights="imagenet",
    input_shape=(300,300,3)
)
```

```
base_model.trainable = False # freeze layer dulu
```

```
inputs = layers.Input(shape=(300,300,3))
x = data_augmentation(inputs)
x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(25, activation="softmax")(x)

model = tf.keras.Model(inputs, outputs)
```

```
model.summary()
```

Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 300, 300, 3)	0
sequential_1 (Sequential)	(None, 300, 300, 3)	0
efficientnetb3 (Functional)	(None, 10, 10, 1536)	10,783,535
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1536)	0
dropout (Dropout)	(None, 1536)	0
dense (Dense)	(None, 25)	38,425

Total params: 10,821,960 (41.28 MB)

Trainable params: 38,425 (150.10 KB)

Non-trainable params: 10,783,535 (41.14 MB)

```
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=12,
    callbacks=[
        tf.keras.callbacks.EarlyStopping(
            patience=3,
            restore_best_weights=True
        )
    ]
)
```

```
Epoch 1/12
200/200 ————— 62s 307ms/step - accuracy: 0.8726 - loss: 0.4783 - val_accuracy: 0.8625 - val_loss: 0.5161
Epoch 2/12
200/200 ————— 82s 306ms/step - accuracy: 0.8742 - loss: 0.4523 - val_accuracy: 0.8637 - val_loss: 0.5032
Epoch 3/12
200/200 ————— 82s 306ms/step - accuracy: 0.8828 - loss: 0.4154 - val_accuracy: 0.8669 - val_loss: 0.4799
Epoch 4/12
200/200 ————— 82s 307ms/step - accuracy: 0.8919 - loss: 0.3882 - val_accuracy: 0.8662 - val_loss: 0.4793
Epoch 5/12
200/200 ————— 61s 306ms/step - accuracy: 0.9017 - loss: 0.3675 - val_accuracy: 0.8725 - val_loss: 0.4660
Epoch 6/12
200/200 ————— 61s 304ms/step - accuracy: 0.9045 - loss: 0.3522 - val_accuracy: 0.8712 - val_loss: 0.4661
Epoch 7/12
200/200 ————— 70s 349ms/step - accuracy: 0.9069 - loss: 0.3388 - val_accuracy: 0.8725 - val_loss: 0.4578
Epoch 8/12
200/200 ————— 70s 350ms/step - accuracy: 0.9066 - loss: 0.3282 - val_accuracy: 0.8763 - val_loss: 0.4516
Epoch 9/12
200/200 ————— 70s 349ms/step - accuracy: 0.9083 - loss: 0.3186 - val_accuracy: 0.8769 - val_loss: 0.4527
Epoch 10/12
200/200 ————— 74s 307ms/step - accuracy: 0.9096 - loss: 0.3140 - val_accuracy: 0.8775 - val_loss: 0.4452
Epoch 11/12
200/200 ————— 61s 305ms/step - accuracy: 0.9133 - loss: 0.2970 - val_accuracy: 0.8806 - val_loss: 0.4406
Epoch 12/12
200/200 ————— 61s 306ms/step - accuracy: 0.9183 - loss: 0.2891 - val_accuracy: 0.8800 - val_loss: 0.4367
```

```
# Menampilkan akurasi terbaik
best_train_acc = max(history.history["accuracy"])
best_val_acc = max(history.history["val_accuracy"])

print("Akurasi Training Terbaik :", best_train_acc)
print("Akurasi Validasi Terbaik :", best_val_acc)
print("Akurasi Keseluruhan Model :", best_val_acc)
```

```
Akurasi Training Terbaik : 0.9207812547683716
Akurasi Validasi Terbaik : 0.8806250095367432
Akurasi Keseluruhan Model : 0.8806250095367432
```

```

test_dataset = tf.keras.utils.image_dataset_from_directory(
    dataset_path,
    image_size=(300, 300),
    batch_size=32,
    shuffle=False,
    label_mode="int"    # ← WAJIB untuk sparse_categorical_crossentropy
)

loss, acc = model.evaluate(test_dataset)
print("Akurasi keseluruhan:", acc)

```

Found 8000 files belonging to 25 classes.
250/250 ————— 57s 226ms/step - accuracy: 0.9221 - loss: 0.2846
 Akurasi keseluruhan: 0.9223750233650208

```

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

# --- 1. Siapkan list untuk menyimpan label asli & prediksi ---
y_true = []
y_pred = []

for images, labels in test_dataset:
    preds = model.predict(images)
    y_pred.extend(np.argmax(preds, axis=1))    # prediksi
    y_true.extend(labels.numpy())             # label asli

# --- 2. Buat confusion matrix ---
cm = confusion_matrix(y_true, y_pred)

# --- 3. Ambil nama kelas ---
class_names = train_ds.class_names

# --- 4. Plot heatmap ---
plt.figure(figsize=(16, 12))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=class_names,
    yticklabels=class_names
)

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix (EfficientNetB3 - Indian Food Classification)")
plt.xticks(rotation=90)
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

```


1/1 ————— 0s 363ms/step
1/1 ————— 0s 265ms/step
1/1 ————— 0s 292ms/step
1/1 ————— 0s 269ms/step
1/1 ————— 0s 311ms/step
1/1 ————— 0s 314ms/step
1/1 ————— 0s 319ms/step
1/1 ————— 0s 305ms/step
1/1 ————— 0s 298ms/step
1/1 ————— 0s 267ms/step
1/1 ————— 0s 291ms/step
1/1 ————— 0s 293ms/step
1/1 ————— 0s 288ms/step
1/1 ————— 0s 281ms/step
1/1 ————— 0s 284ms/step
1/1 ————— 0s 277ms/step
1/1 ————— 0s 296ms/step
1/1 ————— 0s 267ms/step
1/1 ————— 0s 291ms/step
1/1 ————— 0s 275ms/step
1/1 ————— 0s 280ms/step
1/1 ————— 0s 293ms/step
1/1 ————— 0s 305ms/step
1/1 ————— 0s 290ms/step
1/1 ————— 0s 290ms/step
1/1 ————— 0s 311ms/step
1/1 ————— 0s 306ms/step
1/1 ————— 0s 302ms/step
1/1 ————— 0s 312ms/step
1/1 ————— 0s 301ms/step
1/1 ————— 0s 321ms/step
1/1 ————— 0s 318ms/step
1/1 ————— 0s 319ms/step
1/1 ————— 0s 289ms/step
1/1 ————— 0s 306ms/step
1/1 ————— 0s 289ms/step
1/1 ————— 0s 339ms/step
1/1 ————— 0s 346ms/step
1/1 ————— 0s 287ms/step
1/1 ————— 0s 347ms/step
1/1 ————— 0s 300ms/step
1/1 ————— 0s 285ms/step
1/1 ————— 0s 319ms/step
1/1 ————— 0s 289ms/step
1/1 ————— 0s 299ms/step
1/1 ————— 0s 316ms/step
1/1 ————— 0s 277ms/step
1/1 ————— 0s 296ms/step
1/1 ————— 0s 324ms/step
1/1 ————— 0s 278ms/step
1/1 ————— 0s 300ms/step
1/1 ————— 0s 306ms/step
1/1 ————— 0s 284ms/step
1/1 ————— 0s 307ms/step
1/1 ————— 0s 293ms/step
1/1 ————— 0s 308ms/step
1/1 ————— 0s 301ms/step
1/1 ————— 0s 286ms/step
1/1 ————— 0s 321ms/step
1/1 ————— 0s 286ms/step
1/1 ————— 0s 301ms/step
1/1 ————— 0s 284ms/step
1/1 ————— 0s 269ms/step
1/1 ————— 0s 281ms/step
1/1 ————— 0s 298ms/step
1/1 ————— 0s 301ms/step
1/1 ————— 0s 315ms/step
1/1 ————— 0s 297ms/step
1/1 ————— 0s 304ms/step
1/1 ————— 0s 322ms/step
1/1 ————— 0s 289ms/step
1/1 ————— 0s 293ms/step
1/1 ————— 0s 313ms/step
1/1 ————— 0s 349ms/step
1/1 ————— 0s 284ms/step
1/1 ————— 0s 292ms/step
1/1 ————— 0s 283ms/step
1/1 ————— 0s 287ms/step
1/1 ————— 0s 293ms/step
1/1 ————— 0s 292ms/step
1/1 ————— 0s 296ms/step
1/1 ————— 0s 297ms/step
1/1 ————— 0s 300ms/step
1/1 ————— 0s 293ms/step

1/1 ————— 0s 306ms/step
1/1 ————— 0s 301ms/step
1/1 ————— 0s 290ms/step
1/1 ————— 0s 311ms/step
1/1 ————— 0s 286ms/step
1/1 ————— 0s 295ms/step
1/1 ————— 0s 317ms/step
1/1 ————— 0s 316ms/step
1/1 ————— 0s 322ms/step
1/1 ————— 0s 271ms/step
1/1 ————— 0s 300ms/step
1/1 ————— 0s 299ms/step
1/1 ————— 0s 287ms/step
1/1 ————— 0s 299ms/step
1/1 ————— 0s 333ms/step
1/1 ————— 0s 328ms/step
1/1 ————— 0s 313ms/step
1/1 ————— 0s 325ms/step
1/1 ————— 0s 312ms/step
1/1 ————— 0s 289ms/step
1/1 ————— 0s 297ms/step
1/1 ————— 0s 274ms/step
1/1 ————— 0s 270ms/step
1/1 ————— 0s 273ms/step
1/1 ————— 0s 294ms/step
1/1 ————— 0s 272ms/step
1/1 ————— 0s 283ms/step
1/1 ————— 0s 290ms/step
1/1 ————— 0s 286ms/step
1/1 ————— 0s 292ms/step
1/1 ————— 0s 269ms/step
1/1 ————— 0s 277ms/step
1/1 ————— 0s 313ms/step
1/1 ————— 0s 291ms/step
1/1 ————— 0s 294ms/step
1/1 ————— 0s 295ms/step
1/1 ————— 0s 286ms/step
1/1 ————— 0s 314ms/step
1/1 ————— 0s 297ms/step
1/1 ————— 0s 304ms/step
1/1 ————— 0s 303ms/step
1/1 ————— 0s 305ms/step
1/1 ————— 0s 313ms/step
1/1 ————— 0s 293ms/step
1/1 ————— 0s 284ms/step
1/1 ————— 0s 273ms/step
1/1 ————— 0s 292ms/step
1/1 ————— 0s 303ms/step
1/1 ————— 0s 318ms/step
1/1 ————— 0s 373ms/step
1/1 ————— 0s 336ms/step
1/1 ————— 0s 294ms/step
1/1 ————— 0s 272ms/step
1/1 ————— 0s 285ms/step
1/1 ————— 0s 277ms/step
1/1 ————— 0s 276ms/step
1/1 ————— 0s 320ms/step
1/1 ————— 0s 290ms/step
1/1 ————— 0s 297ms/step
1/1 ————— 0s 298ms/step
1/1 ————— 0s 282ms/step
1/1 ————— 0s 292ms/step
1/1 ————— 0s 271ms/step
1/1 ————— 0s 291ms/step
1/1 ————— 0s 272ms/step
1/1 ————— 0s 302ms/step
1/1 ————— 0s 276ms/step
1/1 ————— 0s 286ms/step
1/1 ————— 0s 303ms/step
1/1 ————— 0s 265ms/step
1/1 ————— 0s 297ms/step
1/1 ————— 0s 291ms/step
1/1 ————— 0s 261ms/step
1/1 ————— 0s 273ms/step
1/1 ————— 0s 281ms/step
1/1 ————— 0s 293ms/step
1/1 ————— 0s 319ms/step
1/1 ————— 0s 279ms/step
1/1 ————— 0s 292ms/step
1/1 ————— 0s 319ms/step
1/1 ————— 0s 336ms/step
1/1 ————— 0s 366ms/step
1/1 ————— 0s 343ms/step
1/1 ————— 0s 313ms/step
1/1 ————— 0s 293ms/step

1/1 0s 281ms/step
1/1 0s 292ms/step
1/1 0s 328ms/step
1/1 0s 299ms/step
1/1 0s 302ms/step
1/1 0s 323ms/step
1/1 0s 272ms/step
1/1 0s 288ms/step
1/1 0s 323ms/step
1/1 0s 285ms/step
1/1 0s 277ms/step
1/1 0s 310ms/step
1/1 0s 285ms/step
1/1 0s 289ms/step
1/1 0s 310ms/step
1/1 0s 297ms/step
1/1 0s 266ms/step
1/1 0s 284ms/step
1/1 0s 316ms/step
1/1 0s 290ms/step
1/1 0s 298ms/step
1/1 0s 284ms/step
1/1 0s 291ms/step
1/1 0s 289ms/step
1/1 0s 292ms/step
1/1 0s 264ms/step
1/1 0s 342ms/step
1/1 0s 367ms/step
1/1 0s 291ms/step
1/1 0s 313ms/step
1/1 0s 374ms/step
1/1 0s 296ms/step
1/1 0s 293ms/step
1/1 0s 282ms/step
1/1 0s 292ms/step
1/1 0s 284ms/step
1/1 0s 290ms/step
1/1 0s 308ms/step
1/1 0s 294ms/step
1/1 0s 296ms/step
1/1 0s 303ms/step
1/1 0s 275ms/step
1/1 0s 278ms/step
1/1 0s 277ms/step
1/1 0s 276ms/step
1/1 0s 282ms/step
1/1 0s 293ms/step
1/1 0s 270ms/step
1/1 0s 294ms/step
1/1 0s 308ms/step
1/1 0s 270ms/step
1/1 0s 311ms/step
1/1 0s 315ms/step
1/1 0s 310ms/step
1/1 0s 284ms/step
1/1 0s 301ms/step
1/1 0s 284ms/step
1/1 0s 284ms/step
1/1 0s 307ms/step
1/1 0s 372ms/step
1/1 0s 384ms/step
1/1 0s 381ms/step
1/1 0s 321ms/step
1/1 0s 298ms/step
1/1 0s 299ms/step
1/1 0s 299ms/step
1/1 0s 296ms/step
1/1 0s 315ms/step
1/1 0s 299ms/step
1/1 0s 294ms/step
1/1 0s 314ms/step
1/1 0s 295ms/step
1/1 0s 296ms/step
1/1 0s 303ms/step
1/1 0s 301ms/step
1/1 0s 323ms/step
1/1 0s 287ms/step
1/1 0s 280ms/step
1/1 0s 272ms/step
1/1 0s 267ms/step
1/1 0s 267ms/step

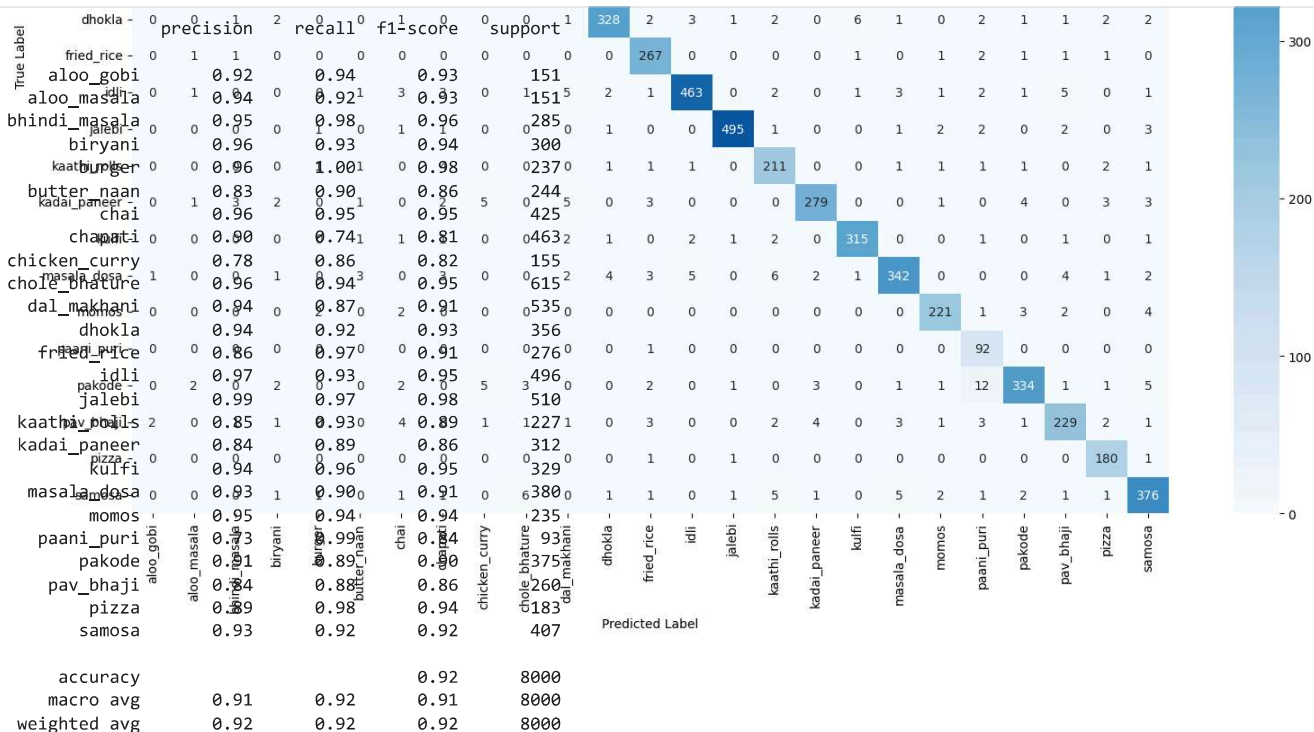
Confusion Matrix (EfficientNetB3 - Indian Food Classification)

aloo_gobi - 142	3	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	4	0	0	0	0	0	0	0
aloo_masala - 6	139	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	5	0	0	0	0	1	0	0

```
from sklearn.metrics import classification_report

# Pastikan class_names sudah ada
class_names = train_ds.class_names

# Print classification report
print(classification_report(
    y_true,
    y_pred,
    target_names=class_names
))
```



```
from sklearn.metrics import classification_report
import pandas as pd
import matplotlib.pyplot as plt

# --- Ambil nama kelas ---
class_names = train_ds.class_names

# --- Buat classification report dalam bentuk dictionary ---
report = classification_report(
    y_true,
    y_pred,
    target_names=class_names,
    output_dict=True
)

# --- Masukkan ke dataframe ---
df = pd.DataFrame(report).transpose()

# Ambil hanya baris kelas (buang accuracy, macro avg, dst)
df_classes = df.iloc[:-3]

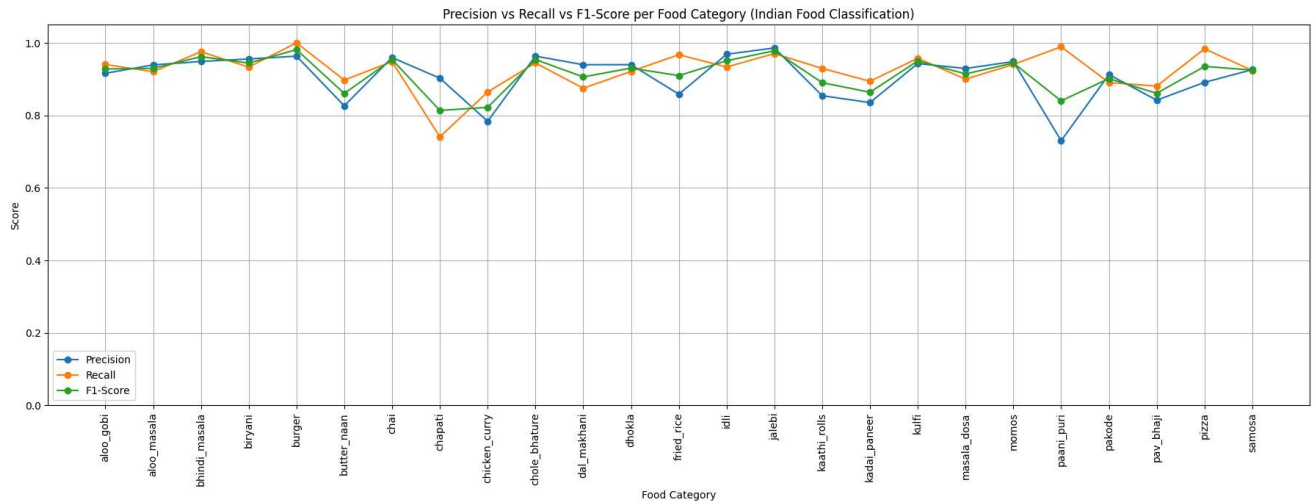
# Ambil nilai precision, recall, f1-score
precision = df_classes["precision"].values
recall = df_classes["recall"].values
f1_score = df_classes["f1-score"].values

food_labels = df_classes.index.tolist()

# --- Plot grafik ---
plt.figure(figsize=(18, 7))
plt.plot(food_labels, precision, marker='o', label='Precision')
plt.plot(food_labels, recall, marker='o', label='Recall')
```

```
plt.plot(food_labels, f1_score, marker='o', label='F1-Score')

plt.title("Precision vs Recall vs F1-Score per Food Category (Indian Food Classification)")
plt.xlabel("Food Category")
plt.ylabel("Score")
plt.xticks(rotation=90)
plt.ylim(0.0, 1.05)
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```



```
import matplotlib.pyplot as plt

acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']

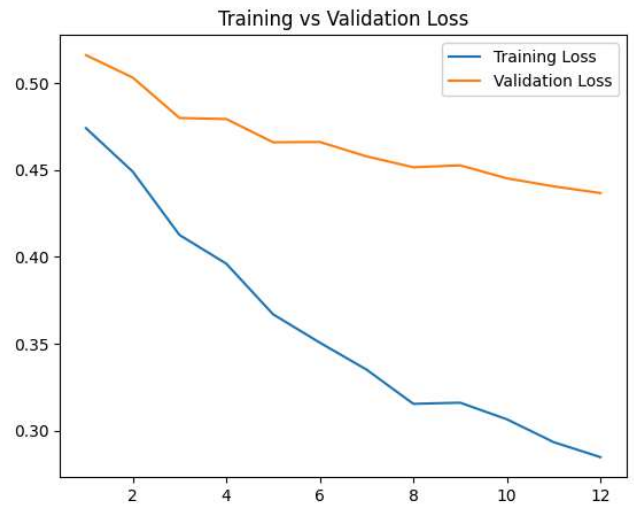
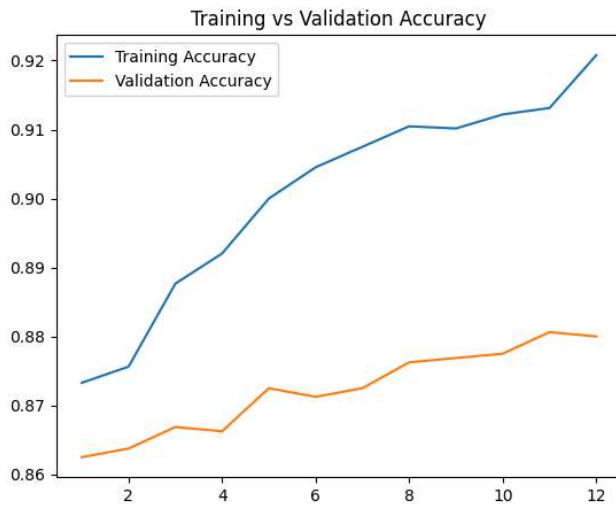
epochs_range = range(1, len(acc)+1)

plt.figure(figsize=(14, 5))

plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label="Training Accuracy")
plt.plot(epochs_range, val_acc, label="Validation Accuracy")
plt.legend()
plt.title("Training vs Validation Accuracy")

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label="Training Loss")
plt.plot(epochs_range, val_loss, label="Validation Loss")
plt.legend()
plt.title("Training vs Validation Loss")

plt.show()
```



```
import os
```

```
base = "/content/IndianFoods25"
for cls in os.listdir(base):
    print("Kelas:", cls)
    sample = os.listdir(os.path.join(base, cls))[:3] # tampilkan 3 gambar pertama
    print("Contoh gambar:", sample)
    print("-"*30)
```

```
Contoh gambar: ['train-00001-of-00003.parquet-3044.jpg', 'train-00001-of-00003.parquet-2933.jpg', 'train-00001-of-00003.parquet-2933.jpg']
-----
Kelas: biryani
Contoh gambar: ['train-00000-of-00003.parquet-765.jpg', 'train-00000-of-00003.parquet-836.jpg', 'train-00000-of-00003.parquet-836.jpg']
-----
Kelas: jalebi
Contoh gambar: ['train-00000-of-00003.parquet-49.jpg', 'train-00000-of-00003.parquet-584.jpg', 'train-00000-of-00003.parquet-584.jpg']
-----
Kelas: kulfi
Contoh gambar: ['train-00001-of-00003.parquet-1198.jpg', 'train-00001-of-00003.parquet-1402.jpg', 'train-00001-of-00003.parquet-1402.jpg']
-----
Kelas: burger
Contoh gambar: ['train-00002-of-00003.parquet-849.jpg', 'train-00002-of-00003.parquet-902.jpg', 'train-00002-of-00003.parquet-902.jpg']
-----
Kelas: aloo_gobi
Contoh gambar: ['train-00001-of-00003.parquet-3939.jpg', 'train-00001-of-00003.parquet-3971.jpg', 'train-00001-of-00003.parquet-3971.jpg']
-----
Kelas: kadai_paneer
Contoh gambar: ['train-00001-of-00003.parquet-2492.jpg', 'train-00001-of-00003.parquet-2599.jpg', 'train-00001-of-00003.parquet-2599.jpg']
-----
Kelas: pav_bhaji
Contoh gambar: ['test-00000-of-00001.parquet-687.jpg', 'train-00002-of-00003.parquet-624.jpg', 'train-00002-of-00003.parquet-624.jpg']
-----
Kelas: dal_makhani
Contoh gambar: ['train-00001-of-00003.parquet-200.jpg', 'train-00001-of-00003.parquet-27.jpg', 'train-00000-of-00003.parquet-4277.jpg']
-----
Kelas: chole_bhature
Contoh gambar: ['validation-00000-of-00001.parquet-672.jpg', 'train-00001-of-00003.parquet-4277.jpg', 'train-00002-of-00003.parquet-624.jpg']
-----
```

```

Contoh gambar: ['test-00000-of-00001.parquet-810.jpg', 'train-00002-of-00003.parquet-1952.jpg', 'train-00002-of-00003.parquet-']
-----
Kelas: fried_rice
Contoh gambar: ['train-00002-of-00003.parquet-4246.jpg', 'train-00002-of-00003.parquet-4241.jpg', 'train-00002-of-00003.parquet-']
-----
Kelas: paani_puri
Contoh gambar: ['train-00001-of-00003.parquet-1554.jpg', 'train-00001-of-00003.parquet-1548.jpg', 'train-00001-of-00003.parquet-']
-----
Kelas: chicken_curry
Contoh gambar: ['train-00000-of-00003.parquet-3362.jpg', 'train-00000-of-00003.parquet-3458.jpg', 'train-00000-of-00003.parquet-']
-----
Kelas: pizza

```

```
img_path = "/content/IndianFoods25/biryani/biryani_23.jpg"
```

```

import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.applications.efficientnet import preprocess_input

# Ambil label otomatis
class_names = train_ds.class_names

def load_and_preprocess(img_path):
    img = tf.keras.utils.load_img(img_path, target_size=(300, 300))
    img_array = tf.keras.utils.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0)
    img_array = preprocess_input(img_array)
    return img, img_array

# GANTI path ini sesuai file asli dalam folder IndianFoods25
img_path = "/content/IndianFoods25/chai/test-00000-of-00001.parquet-225.jpg"

img, img_array = load_and_preprocess(img_path)

pred = model.predict(img_array)
pred_class = class_names[np.argmax(pred)]
pred_prob = np.max(pred) * 100

plt.imshow(img)
plt.axis("off")
plt.title(f"Prediksi: {pred_class} ({pred_prob:.2f}%)" )
plt.show()

```

1/1 ————— 0s 51ms/step

Prediksi: chai (99.53%)



```
!zip -r indianfoods25.zip /content/IndianFoods25
```

```

adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3724.jpg (deflated 1%)
adding: content/IndianFoods25/idli/test-00000-of-00001.parquet-566.jpg (deflated 1%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3371.jpg (deflated 1%)
adding: content/IndianFoods25/idli/validation-00000-of-00001.parquet-600.jpg (deflated 0%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3646.jpg (deflated 1%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3637.jpg (deflated 1%)
adding: content/IndianFoods25/idli/test-00000-of-00001.parquet-544.jpg (deflated 3%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3770.jpg (deflated 1%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3501.jpg (deflated 2%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3748.jpg (deflated 1%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3736.jpg (deflated 0%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3619.jpg (deflated 0%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3473.jpg (deflated 0%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3737.jpg (deflated 1%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3531.jpg (deflated 2%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3809.jpg (deflated 1%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3410.jpg (deflated 1%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3553.jpg (deflated 3%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3805.jpg (deflated 1%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3740.jpg (deflated 1%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3494.jpg (deflated 1%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3611.jpg (deflated 0%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3842.jpg (deflated 0%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3778.jpg (deflated 1%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3454.jpg (deflated 0%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3673.jpg (deflated 2%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3777.jpg (deflated 1%)
adding: content/IndianFoods25/idli/validation-00000-of-00001.parquet-587.jpg (deflated 0%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3821.jpg (deflated 0%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3496.jpg (deflated 1%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3304.jpg (deflated 4%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3442.jpg (deflated 1%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3416.jpg (deflated 1%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3445.jpg (deflated 3%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3558.jpg (deflated 2%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3520.jpg (deflated 2%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3484.jpg (deflated 2%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3838.jpg (deflated 0%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3575.jpg (deflated 5%)
adding: content/IndianFoods25/idli/validation-00000-of-00001.parquet-564.jpg (deflated 0%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3739.jpg (deflated 1%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3803.jpg (deflated 7%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3653.jpg (deflated 0%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3524.jpg (deflated 2%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3349.jpg (deflated 0%)
adding: content/IndianFoods25/idli/validation-00000-of-00001.parquet-605.jpg (deflated 1%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3345.jpg (deflated 1%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3418.jpg (deflated 1%)
adding: content/IndianFoods25/idli/test-00000-of-00001.parquet-586.jpg (deflated 7%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3835.jpg (deflated 0%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3807.jpg (deflated 1%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3642.jpg (deflated 0%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3630.jpg (deflated 3%)
adding: content/IndianFoods25/idli/train-00001-of-00003.parquet-3644.jpg (deflated 0%)

```

```

from google.colab import files
files.download("indianfoods25.zip")

```

```

# 1. Buka (unfreeze) seluruh EfficientNetB3
base_model.trainable = True

# 2. Compile ulang dengan learning rate kecil
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5), # SUPER PENTING!
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# 3. Training lagi
history_ft = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=10,
    callbacks=[
        tf.keras.callbacks.EarlyStopping(
            monitor="val_accuracy",
            patience=3,
            restore_best_weights=True
        )
    ]
)

```